

Bridge Bidding Platform Software Engineering Implementation Document

Platform Version: 0.0.4

Author: Meow Li

Copyright: Copyright (c) 2026 Meow Li. All Rights Reserved.

This document describes the current checkpoint implementation for engineers and AI agents. It should be detailed enough to inspect the code file by file and recover the active design from a fresh start.

0. Documentation Map

Read ``docs/documentation_system.md`` first. Companion documents:

? ``docs/product_description.md``: highest-layer product description.

? ``docs/user_manual.md``: user guide and gadget-authoring guide.

? ``docs/semantic_ontology.md``: formal bridge-domain semantic vocabulary and target executable behavior.

? ``docs/bridge_system_language_roadmap.md``: roadmap for BSL, IR/YAML, GUI authoring, compiler, and interpreter layers.

? ``docs/todo.md``: future work, open issues, and deferred ideas.

Markdown is the editable source of truth. PDFs in ``docs/pdf/`` are regenerated only for releases or explicit PDF-generation requests.

1. Repository Layout

```
backend/  
  app.py  
  engine/  
    __init__.py  
    auction.py  
    calls.py  
    cards.py  
    evaluator.py  
    explanation.py  
    gadget.py  
    loader.py  
    matcher.py  
    selector.py  
    trace.py  
  gadgets/  
    two_over_one/  
      gadget.yaml  
      default_policy.yaml  
      openings_notrump.yaml  
      openings_one_level.yaml  
      openings_two_level.yaml  
    four_way_jacoby_transfer/  
      gadget.yaml  
      notrump_responses.yaml  
      continuations_hearts.yaml  
  systems/  
    expert_2over1.yaml  
  tests/  
    README.md
```

```

    test_cases.md
    test_fixture_cases.py
    cases/
        bidding.yaml
        matcher.yaml
docs/
    bridge_system_language_roadmap.md
    documentation_system.md
    implementation_design.md
    product_description.md
    semantic_ontology.md
    todo.md
    user_manual.md
    pdf/

```

There is no production API server yet. `backend/app.py` exposes the current Python function boundary and a command-line sample.

1.1 Target Authoring Architecture

Current implementation loads YAML gadget rules directly. The target architecture separates authoring from execution:

```

Form UI and BSL source
-> compiler and validator
-> normalized IR/YAML
-> engine interpreter

```

The engine should execute only validated intermediate representation. YAML is currently the practical serialization format for that representation. The Bridge System Language is a future formal authoring language, not loose English. See `docs/bridge\_system\_language\_roadmap.md` for the staged roadmap.

2. Execution Flow

The current bidding path is:

```

backend/app.py bid(request)
-> loader.load_system(system_id, backend_dir)
    -> read backend/systems/<system_id>.yaml
    -> load each gadget directory
    -> merge rules from every YAML file in that gadget directory
-> Auction.parse(request["auction"], dealer, vulnerability)
-> Hand.parse(request["hand"])
-> selector.choose_bid(rules, auction, hand, environment)
    -> replay_auction(rules, auction, hand, environment)
        -> interpret historical non-pass calls through rules with meaning
        -> add semantic facts to SemanticTrace
    -> search every active gadget rule with selection
    -> exact-match current auction pattern
    -> evaluate top-level applicability
    -> evaluate selection.applicability
    -> evaluate structured criteria
    -> compare Candidate objects by score
    -> if none match, use an explicit default_policy rule
-> explanation.explain(selection)
-> return dict with call, public_meaning, internal_origin, diagnostics

```

Important architecture rule: all active gadgets contribute in parallel over the auction state. A rule after `1NP` may live in `four\_way\_jacoby\_transfer` even though the `1N` opening lives in `two\_over\_one`; the selector searches all active gadgets before defaulting.

3. Canonical Call System

Implemented in ``backend/engine/calls.py``.

Canonical calls:

```
P = Pass
X = Double
R = Redouble
1C..7C = clubs contracts
1D..7D = diamonds contracts
1H..7H = hearts contracts
1S..7S = spades contracts
1N..7N = notrump contracts
```

The formal field name is ``suit``, and the canonical suit order is:

C < D < H < S < N

``N`` represents notrump in the formal ``suit`` field.

``ContractBid``

Frozen dataclass fields:

? ``level: int``: ``1`` through ``7``.

? ``suit: str``: one of ``C D H S N``.

Methods:

? ``parse(call)``: normalizes input, rejects ``P X R``, and returns a contract object.

? ``rank``: ``(level - 1) * len(SUITS) + SUIT_RANK[suit]``.

? ``__str__``: returns canonical text such as ``1N``.

`Functions`

? ``normalize_call(call)``: strips, uppercases, maps aliases (``PASS``, ``DBL``, ``XX``, ``1NT``), validates, and returns canonical text.

? ``parse_call_sequence(sequence)``: parses compact auction strings such as ``1NP1SPXR``, allowing spaces anywhere.

? ``normalize_pattern(pattern)``: normalizes compact auction patterns and preserves the special default wildcard ``*``.

? ``is_contract_bid(call)``: true for canonical two-character contract calls.

? ``compare_contract_bids(left, right)``: compares by contract rank.

4. Auction Model

Implemented in ``backend/engine/auction.py``.

Constants:

? ``SEATS`` = `("n", "e", "s", "w")``

? ``VULNERABILITY_STATES = ("none", "ns", "ew", "both")``

``Auction`` is a frozen dataclass with:

? ``calls: tuple[str, ...]``: canonical calls.

? ``dealer: str``: absolute dealer.

? ``vulnerability: str``: absolute vulnerability state.

Methods:

? ``parse(calls, dealer="n", vulnerability="none")``: validates dealer/vulnerability and normalizes calls.

? ``actor_to_call``: seat whose turn it is.

? ``actor_at(index)``: absolute seat for an auction index.

? ``compact_sequence()``: compact call sequence such as ``1NP2D``.

? ``canonical_key()``: unique representation such as ``dealer=n;vul=none;calls=1NP2D``.

Current limitation: token form is validated, but full bridge auction legality is not yet implemented.

5. Hand Model

Implemented in ``backend/engine/cards.py``.

``Hand`` is a frozen dataclass with:

? ``spades: str``

? ``hearts: str``

? ``diamonds: str``

? ``clubs: str``

Each stores uppercase rank text. ``10`` is normalized to ``T``, and ``X`` stores an unknown small-card placeholder.

Properties and methods:

? ``parse(data, validate_count=True)``: accepts a compact string.

? ``from_compact(text, validate_count=True)``: parses raw user input such as ``H8763SK10C2DAKQ987``.

? ``from_dict(data, validate_count=False)``: reads four long-form suit keys, missing keys become empty strings.

? ``validate(validate_count=True)``: checks repeated known cards and optionally checks total card count.

? ``hcp``: computes ``A=4, K=3, Q=2, J=1``.

? ``suits``: returns all four suit strings.

? ``length(suit)``: accepts long-form suit names such as ``hearts``.

? ``balanced``: true for ``5332``, ``4432``, or ``4333``.

Raw hand input is always a compact string. The internal ``Hand`` object still stores one field per suit and exposes a dictionary through ``Hand.suits``.

Compact hand parser behavior:

- ? Suit markers are `S H D C`.
- ? Suit order is flexible.
- ? Repeated suit sections are concatenated, so `S9S8` means spades `98`.
- ? Missing suits are treated as void if the total card count is still correct.
- ? `` marks a void suit section.
- ? `10` and `T` both mean ten.
- ? `x` or `X` is accepted as an unknown small card placeholder.
- ? Raw compact input validates that the hand has 13 cards.
- ? Known physical cards cannot repeat, such as `SASAH...`.
- ? Unknown symbols, rank text before any suit marker, isolated `1`, and `` mixed with cards raise `ValueError`.

6. Gadget Model

Implemented in `backend/engine/gadget.py`.

The backend has one technical module type: `Gadget`. User-facing UI may call `two\_over\_one` a basic system, base system, or foundational agreement set, but the backend does not create a separate class or schema type for that distinction.

`Author`

Frozen dataclass fields:

- ? `name: str`
- ? `contact: str | None`
- ? `organization: str | None`

`from\_dict` defaults missing names to `Unknown`. `to\_dict` preserves all three fields.

`Gadget`

Frozen dataclass fields:

- ? `id: str`: short stable ID, such as `four\_way\_jacoby\_transfer`.
- ? `namespace: str`: origin namespace, such as `notrump\_response`.
- ? `name: str`: human-readable name.
- ? `version: str`: gadget version, defaulting to `0.1.0` if omitted.
- ? `author: Author`: default author for rules.
- ? `rules: tuple[GadgetRule, ...]`: immutable loaded rules.

Constructors:

- ? `from\_dict(data)`: compatibility constructor for old single-file gadgets.

? ``from_parts(metadata, rule_data)``: directory-gadget constructor using ``gadget.yaml`` metadata plus rule YAML files.

``qualified_id`` format:

namespace/id@version

``GadgetRule``

Frozen dataclass fields:

? ``id: str``: short rule ID.

? ``context: dict[str, Any]``: rule context, primarily ``auction_pattern`` plus optional ``seat_positions``.

? ``call: str``: canonical call.

? ``gadget_id: str``: owning gadget ID.

? ``gadget_namespace: str``: owning namespace.

? ``gadget_version: str``: owning gadget version.

? ``gadget_name: str``: owning gadget name.

? ``author: Author``: rule author, inherited unless overridden.

? ``applicability: dict[str, Any]``: context/environment precondition.

? ``selection: dict[str, Any]``: current-hand bid-selection algorithm and criteria.

? ``meaning: dict[str, Any]``: public partnership meaning and alertability metadata.

? ``semantic_effects: tuple[dict[str, Any], ...]``: facts emitted during replay.

? ``default_policy: bool``: true for explicit fallback rules.

Properties:

? ``auction_pattern``: returns ``context.auction_pattern`` when present, otherwise an empty string.

? ``has_selection``: true when ``selection`` is non-empty.

? ``has_meaning``: true when ``meaning`` is non-empty.

? ``qualified_rule_id``: ``namespace/gadget_id@version:rule_id``.

Current schema rule: a rule is unified by content. A rule may have ``selection``, ``meaning``, and ``semantic_effects`` together.

Field distinction:

? Top-level ``applicability`` is used during historical replay and current selection. It should contain conditions valid for the auction context.

? ``selection.applicability`` is used only when deciding whether the current hand should choose this call. Replay does not use it, because replay may be interpreting another player's earlier hand.

? ``context.seat_positions`` expands an ``auction_pattern`` by prepending the initial passes for the numbered seats. For example, ``auction_pattern: "1HP"`` with ``seat_positions: [3, 4]`` matches ``PP1HP`` and ``PPP1HP``.

Compatibility behavior: top-level ``auction_pattern`` is accepted as input and normalized into ``context.auction_pattern``.

``origin_dict()`` returns rule/gadget/author metadata for public meaning, internal origin, semantic facts, and applied meaning rules.

7. Semantic Trace

Implemented in ``backend/engine/trace.py``.

The current trace is a prototype. The system-level target is defined in ``docs/semantic_ontology.md``. Future engine work should replace ad-hoc fact strings with ontology-backed semantic state for concepts such as transfers, agreed suits, forcing obligations, competitive interference, notrump focus, and keycard context.

``SemanticFact`` fields:

? ``fact_type: str``
? ``attributes: dict[str, Any]``
? ``origin: dict[str, Any]``

Methods:

? ``from_dict(data, origin)``: extracts ``fact_type`` and stores remaining keys as attributes.
? ``matches(query)``: exact-match query over ``fact_type`` and attributes.
? ``to_dict()``: flattens facts for output.

``SemanticTrace`` fields:

? ``facts: list[SemanticFact]``
? ``applied_meaning_rules: list[dict[str, Any]]``
? ``diagnostics: list[str]``

Methods append facts, query facts, record applied meaning rules, and add warnings.

Current limitation: ``SemanticFact.fact_type`` and its attributes are flexible dictionaries. This is enough for the first Jacoby-transfer sample, but it does not yet enforce the formal ontology. Portable gadgets should converge toward the shared vocabulary in ``docs/semantic_ontology.md`` instead of inventing private fact names that other gadgets cannot query safely.

8. Pattern Matching

Implemented in ``backend/engine/matcher.py``.

? ``matches_context(context, auction)``: matches a full rule context. It supports exact ``auction_pattern`` plus optional ``seat_positions`` expansion.
? ``matches_pattern(pattern, auction, seat_positions=None)``: normalizes ``pattern``, expands it for the requested seat positions, and compares each expansion to the current auction calls.
? ``historical_pattern_for(calls, index)``: returns compact calls before ``index``.

Exact ``auction_pattern`` matching remains the default. Seat-position expansion is a general shorthand for initial-pass variants, so it works for openings, Drury-style auctions, passed-hand responses, and continuations. The default-policy wildcard ``*`` is handled by ``selector._default_candidate``.

9. Evaluation

Implemented in ``backend/engine/evaluator.py``.

``evaluate(condition, hand, trace, environment=None)`` supports:

- ? ``all``
- ? ``any``
- ? ``not``
- ? ``fact_exists``
- ? direct comparisons against resolved values.

Resolved values:

- ? ``self.hcp``
- ? ``self.balanced``
- ? ``self.<long_suit_name>``, for example ``self.hearts``
- ? ``env.<key>``

Comparison keys:

- ? ``min``
- ? ``max``
- ? ``eq``
- ? ``in``

``evaluate_selection(selection, hand, trace, environment=None)`` currently supports one algorithm:

`weighted_score`

It evaluates every criterion, marks the rule ineligible if a required criterion fails, sums weights for passed criteria, and returns:

```
{
  "algorithm": algorithm,
  "eligible": eligible,
  "score": score,
  "criteria_results": criteria_results,
}
```

``evaluate_criterion`` supports:

- ? ``range_contains``
- ? ``min_value``
- ? ``equals``
- ? ``fact_exists``

Criteria are required by default. Add ``required: false`` for optional scoring criteria.

10. Selection

Implemented in ``backend/engine/selector.py``.

``Candidate`` fields:

? ``call``
?
?``origin``
?``public_meaning``
?``algorithm``
?``rule_id``
?``score``
?``criteria_results``

``Selection`` fields:

? ``call``
?``selected``
?``candidates``
?``trace``

``choose_bid(rules, auction, hand, environment=None)``:

1. Calls ``replay_auction``.
2. Iterates every active rule from every loaded gadget.
3. Skips default policies during normal candidate search.
4. Requires ``rule.has_selection``.
5. Requires exact auction-pattern match.
6. Requires top-level ``applicability``.
7. Requires ``selection.applicability``.
8. Converts eligible rules to candidates.
9. Sorts by score descending.
10. Warns on tied top score.
11. Uses explicit default-policy rule only when no normal candidate exists.

``replay_auction(rules, auction, hand=None, environment=None)``:

1. Iterates historical calls.
2. Skips ``P``.
3. Builds the pattern before each historical call.
4. Finds rules with ``meaning``, matching context, and matching call.
5. If ``hand`` is supplied, filters by top-level ``applicability``.

6. Warns if no rule or multiple rules match.
7. Emits ``semantic_effects``.
8. Records applied meaning rule origin and public meaning.

Default handling:

- ? ``_default_candidate`` searches loaded rules where ``default_policy: true``.
- ? A default rule can use ``context.auction_pattern: "*"`` to match any unmatched current auction.
- ? ``_default_pass_candidate`` is an ultimate hard fallback only if no loaded default-policy rule exists.

11. Explanation Output

Implemented in ``backend/engine/explanation.py``.

``explain(selection)`` returns:

- ? ``call``: selected canonical call.
- ? ``public_meaning``: disclosure layer, containing selected origin and meaning.
- ? ``internal_origin``: training/debug layer.
- ? ``diagnostics``: warnings.

``internal_origin`` contains:

- ? ``selected``
- ? ``compared_candidates``
- ? ``applied_meaning_rules``
- ? ``semantic_facts``

Public meaning and internal origin must stay separate. Internal comparison detail is useful for training and system debugging, but it is not automatically opponent disclosure.

12. Loader

Implemented in ``backend/engine/loader.py``.

Optional dependency:

- ? Uses PyYAML when available.
- ? Falls back to ``json.loads`` if PyYAML is unavailable.

Human-authored files should be real YAML. JSON-compatible YAML remains supported for generated artifacts and fallback portability.

``load_system(system_id, base_dir=None)``:

- ? Reads ``systems/<system_id>.yaml``.

? For each gadget name, builds `base\_dir / "gadgets" / Path(\*gadget\_name.split("."))`.

? Calls `\_load\_gadget`.

? Returns a flat list of `GadgetRule` objects.

`_load_gadget(gadget_path)`:

? If `gadget\_path` is a directory, reads `gadget.yaml` as metadata.

? Reads every other `\*.yaml` file in sorted order.

? Extends one rule list from each file's `rules` array.

? Calls `Gadget.from\_parts(metadata, rule\_data)`.

? If no directory exists, falls back to old `gadget\_path.with\_suffix(".yaml")` single-file loading.

13. Public Function

Implemented in `backend/app.py`.

`bid(request)` expects:

```
{
  "system": {"id": "expert_2over1"},
  "seat": "n",
  "auction": "1NP2DP",
  "hand": "SAQ74HKJ83DA62CQ5",
  "environment": {
    "dealer": "n",
    "vulnerability": "none",
    "scoring": "IMP",
  },
}
```

Local variables:

? `system\_id`

? `environment`

? `dealer`

? `vulnerability`

? `rules`

? `auction`

? `hand`

? `selection`

Returns `explain(selection)`.

14. Current System And Gadgets

`backend/systems/expert\_2over1.yaml` imports:

```
gadgets:
  - two_over_one
```

- four_way_jacoby_transfer

`backend/gadgets/two\_over\_one/`

Metadata file:

? `gadget.yaml`

Rule files:

? `openings\_notrump.yaml`: `open\_1N`, `open\_2N`.

? `openings\_one\_level.yaml`: starter rules for `1H`, `1S`, `1C`, `1D`, with 1/2-seat and placeholder 3/4-seat structure.

? `openings\_two\_level.yaml`: starter rules for `2C`, `2D`, `2H`, `2S`.

? `default\_policy.yaml`: explicit `default\_pass` rule with `context.auction\_pattern: ""` and `default\_policy: true`.

The `two\_over\_one` gadget is a normal backend gadget even though users may perceive it as the basic system.

`backend/gadgets/four\_way\_jacoby\_transfer/`

Metadata file:

? `gadget.yaml`

Rule files:

? `notrump\_responses.yaml`: `transfer\_hearts\_2D`.

? `continuations\_hearts.yaml`: `complete\_hearts`, `superaccept\_hearts`.

`transfer\_hearts\_2D` currently requires at least five hearts and records public meaning as a heart transfer. This is intentionally only a first implementation. Later Texas transfers, Smolen, invitational routes, and game-forcing routes must compete through structured evaluation over hand, auction, environment, and semantic facts.

15. Tests

Backend behavior tests are fixture-driven.

Test files:

? `backend/tests/cases/bidding.yaml`: bridge bidding examples and expected engine results.

? `backend/tests/cases/matcher.yaml`: auction-pattern and seat-position expansion examples.

? `backend/tests/cases/hands.yaml`: compact hand parser examples and expected errors.

? `backend/tests/test\_fixture\_cases.py`: generic Python runner that loads YAML cases.

? `backend/tests/README.md`: workflow for adding cases.

? `backend/tests/test\_cases.md`: human-readable translation of the fixture cases.

The Python runner should stay generic. Add new bridge examples to YAML case files. When changing fixture cases, update `backend/tests/test\_cases.md` in the same checkpoint.

Current covered behavior:

? `1NTP2DP` selects `3H` with a maximum-ish 1N hand and four-card heart support.

? `1NP` selects `2D` with five hearts.

? `""`, `P`, `PP`, and `PPP` can select `1N` through `seat\_positions`.

? `PP1NP` can use the same notrump response rule as `1NP`.

? `PP1NP2DP` can use the same continuation rule as `1NP2DP`.

? `1NP` defaults to pass with only four hearts.

? Unmatched auctions use the loaded default policy and emit diagnostics.

? Calls normalize from aliases such as `1NT`, `pass`, and lowercase suits.

? Loader accepts JSON-compatible YAML through the single-file compatibility path.

? Compact hand parsing accepts suit-order variation, `10`, repeated suit sections, voids, and `X` placeholders.

? Compact hand parsing rejects wrong card counts, repeated known cards, unknown symbols, and malformed void markers.

Run from the workspace with:

```
$env:PYTHONPATH='backend'; C:\Users\paw_l\cache\codex-runtimes\codex-primary-runtime\dependencies\python\python.exe -m unittest discover backend\tests
```

16. Output Contract

Shape:

```
{
  "call": "3H",
  "public_meaning": {
    "origin": {
      "namespace": "notrump_response",
      "gadget_id": "four_way_jacoby_transfer",
      "gadget_name": "Four-Way Jacoby Transfer",
      "gadget_version": "0.1.0",
      "rule_id": "superaccept_hearts",
      "qualified_rule_id": "notrump_response/four_way_jacoby_transfer@0.1.0:
superaccept_hearts",
      "author": {
        "name": "Partner Prototype",
        "contact": null,
        "organization": null
      }
    },
    "meaning": {
      "call_nature": "conventional",
      "action_type": "superaccept",
      "target_suit": "H",
      "alertable": true
    }
  },
  "internal_origin": {
    "selected": {},
    "compared_candidates": [],
    "applied_meaning_rules": [],
    "semantic_facts": []
  },
  "diagnostics": []
}
```

The exact contents of ``internal_origin`` vary by auction and candidate set.

17. Current Technical Limitations

Active limitations:

1. Full bridge auction legality is not implemented.
2. Double/redouble legality and auction completion are not implemented.
3. Seat roles such as opener/responder are not derived generally.
4. Relative concepts such as our side, their side, favorable vulnerability, balancing seat, and 1/2/3/4 seat are not yet computed centrally.
5. Pattern matching is exact except for default-policy ``*``.
6. Only ``weighted_score`` and a small set of built-in evaluators exist.
7. Custom evaluator plugins are not implemented.
8. ACBL alertability is hard-coded through ``meaning.alertable``.
9. The 1NT continuation library is intentionally incomplete.

18. Preserved Technical Requirements

1. The public bidding API should remain stateless. Clients submit visible bridge context.
2. Semantic state must be derived by replaying the auction.
3. Gadgets are convention subsystems, not one-off toggles.
4. Public meaning and internal origin must remain separate.
5. Active gadgets contribute rules in parallel over the auction space.
6. Structured criteria, not natural-language prose, are the source of bidding logic.
7. A basic system is a normal backend gadget with a user-facing label.
8. Default behavior must be explicit and customizable over time.
9. Custom judgment, including choosing between 1N and 1S or between Jacoby, Texas, and Smolen routes, must report rule origins and compared candidates.
10. Future custom evaluation must be practical and restricted through approved algorithms or plugin hooks, not arbitrary prose.
11. Semantic concepts shared across gadgets must use the formal ontology in ``docs/semantic_ontology.md``, so later rules can reason about transfers, agreed trump, forcing status, competitive state, and ``4N`` meanings without enumerating every possible auction pattern.
12. GUI forms, BSL source, and IR/YAML should be treated as different authoring surfaces over the same validated executable rule model. The engine executes IR, not the GUI form state or natural-language text.

19. Documentation Maintenance Rule

When changing code, update this document in the same checkpoint if structure, schemas, behavior, output, or limitations changed. Do not bump the platform version unless the user explicitly says to release a new version.